

s2.03 — Certificate revocation and status distribution — CRL / OCSP

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Revocation ledger and CRL publication — IC3 baseline](#)
- [OCSP responder, stapling, and relying-party policy — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: JD fit plus a preparation addition, recorded in [the bank](#).
 - The architecture below is a proposed interview answer.
 - Documented OCI behavior is marked as such
 - Everything else is proposed design.
- Why this is hard
 - Naive: delete the certificate row and return `404` from a JSON status endpoint.
 - First break: ordinary TLS verifiers do not call a custom status REST endpoint.
 - They need supported authenticated status evidence, and deleting the resource must not erase revocation history.
 - Second break: the signed artifact is cached, so a key revoked at 09:00 is still accepted until every cached CRL or OCSP response reaches `nextUpdate`
 - If the responder is down, verifiers either accept everything (soft-fail) or reject everything (hard-fail).
 - Minimum primitives
 - An append-only ledger with a per-issuer generation
 - Signed CRL snapshots with a CRL number
 - HSM-held signing keys
 - A publish-then-advance pointer
 - An explicit relying-party policy.
- 🗨️ "Revocation is a signed, cached, eventually consistent negative; the design is bounding how stale that negative can be without making the issuer a handshake dependency."
- Opening script
 - "I will treat revocation as an ordered ledger per issuer that feeds two signed status artifacts: CRL snapshots and OCSP responses."

- "First I confirm who the relying parties are, whether they can check online, and the propagation bound the team accepts after a key compromise."
- "Then I size revocation rate, CRL bytes, and OCSP signing load, and define a work-request API where revoking twice is safe."
- "The deep dives are the publish-then-advance invariant for CRLs, and an OCSP responder serving precomputed responses signed by an HSM-held responder key."
- "I will close with static stability during a control-plane outage, soft-fail versus hard-fail, and where short-lived certificates change the answer."

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Relying parties, propagation bound, primary invariant
Estimates and contract	11:00–17:00	Revocation rate, CRL bytes, signing load; API semantics
Architecture	17:00–30:00	Diagram, control/data-plane split, one revocation trace
Deep dive	30:00–45:00	Generation ledger, publish-then-advance, OCSP responder
Operations and tradeoffs	45:00–50:00	Failure table, freshness alarms, soft-fail tradeoff, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Who are the relying parties?	OCI load balancers, gateways, internal mTLS services we control; some appliances and DB wallets that only fetch CRLs	CRL is mandatory for the appliance cohort; OCSP is an addition for the controlled cohort
Accepted propagation bound after key compromise?	≤ 1 h for controlled verifiers; ≤ 24 h for CRL-only consumers	Fixes CRL <code>nextUpdate</code> , OCSP validity, and CDN <code>max-age</code>
Revocation volume?	1M active certificates, $< 1\%$ revoked per year, spikes of 10k in an hour	Sizes CRL bytes; forces debounced republish
Is OCSP on the handshake path?	No for servers we control (they staple); optionally for clients	Responder must be statically stable and serve precomputed responses
Soft-fail or hard-fail?	Owned by the relying party; we publish an SLO that makes hard-fail viable for stapled servers	Sets responder availability target and staple refresh schedule
Multi-tenant?	Yes; every issuer lives in a tenant compartment	Authorization and artifacts are per issuer; never a global CRL

- Functional scope
 - Revoke a certificate by `(issuer, serial)` with a reason and optional invalidity date.
 - Build, sign, and publish a per-issuer CRL on schedule and on demand.
 - Answer OCSP requests per RFC 6960 with a signed status for one `CertID`.
 - Supply fresh OCSP responses to servers we control for stapling.
 - Expose administrative revocation status and the publication generation.
 - Audit every revocation, publication, and signature.

Requirement	Target	Enforced by
Revocation accept latency	p99 < 500 ms to a durable 202	Single-transaction ledger write plus outbox; no signing on the request path
Publication lag	≤ 5 min normal; ≤ 60 s on forced publish	60 s per-issuer debounce; forced publish bypasses it; CDN purge
CRL validity	nextUpdate = thisUpdate + 24 h; republished every 1 h	Publisher schedule; page when nextUpdate - now < 6 h
OCSP response validity	nextUpdate = thisUpdate + 8 h; re-signed every 1 h	Precompute batch per issuer; page when oldest response age > 2 h
OCSP responder availability	99.99% monthly; p99 < 20 ms	Stateless responders on an in-memory snapshot; spread across availability domains
CRL download availability	99.99%; independent of control-plane health	Object storage plus CDN; a signed CRL validates without the service
Static stability	Continue until the earliest required evidence deadline; test 8 h boundary	Cached signed status avoids live fetch only while still acceptable; do not promise a universal 24 h outage
Retention	Revocation rows kept ≥ notAfter + 30 d; audit 7 y	No DELETE grant; archive job gated by the expiry rule
Signing key custody	0 plaintext private-key exports; 100% durable signing intents	HSM sign path and attempt auditing; responder validity ≤ 30 d

- Invariants
 - A durable revocation appears on every later CRL of that issuer until the certificate's notAfter has passed and one scheduled CRL after that has been issued (RFC 5280 §3.3).
 - Revocation generation and CRL number are strictly monotonic per issuer.
 - The public pointer moves only to a snapshot that was fully written, re-downloaded, and signature-verified.
 - OCSP good is produced only from a signed snapshot with a known through_generation
 - A store outage never produces good.
 - Every status artifact is signed by a key that never leaves the HSM.
 - Dropping or archiving a row never turns revoked into good.
- Non-goals
 - certificateHold and removeFromCRL.
 - Browser-vendor push filters such as CRLite or CRLSets.
 - Path validation, validity dates, and hostname checks
 - OCSP good does not replace them.
 - Revoking a root
 - That is a trust-store change handled in s2.02.

Capacity and consequence — IC3 baseline

- Revocation writes
 - 1M × 1% / year ≈ 10k / year ≈ 30 / day
 - Incident spike 10k / h ≈ 3 / s against one issuer.
 - Decision: one serialized generation counter per issuer row is safe
 - The ledger needs no write sharding.
- CRL bytes
 - Each DER entry is roughly 40–60 B: serial up to 20 B, time ≈ 15 B, reason extension ≈ 10 B (order of magnitude).
 - 10k unexpired entries ≈ 0.5 MB
 - 100k ≈ 5 MB.
 - Decision: CRLs are per issuer and entries leave after expiry
 - Delta or partitioned CRLs become a stretch item only past ≈ 10 MB.

- CRL signing
 - 10k issuers × 24 republishes per day ≈ 240k signatures per day ≈ 3 / s.
 - Decision: CRL signing does not size the HSM
 - OCSP precompute does.
- OCSP precompute
 - 1M certificates re-signed every 1 h ≈ 280 signatures per second.
 - At an assumed mean signing time of 2 ms, 280/s implies about 0.56 average signatures in flight.
 - This does not establish per-partition throughput or p99 under contention.
 - Benchmark the selected HSM, algorithm mix, and failure headroom before allocating capacity.
 - Decision: precompute for every unexpired certificate
 - The refresh interval is the HSM sizing knob, and lazy sign-on-miss is only an overflow path.
- OCSP reads
 - Assumed 5k requests per second at peak after stapling absorbs servers
 - ≈ 1.5 KB per response ≈ 7.5 MB/s.
 - Decision: responders read from an in-memory snapshot keyed by CertID
 - No database on the read path.
- CRL fetches
 - 10k consumers refreshing hourly ≈ 3 / s
 - A restart storm of 10k in one minute ≈ 170 / s × 0.5 MB ≈ 85 MB/s.
 - Decision: CDN fronts object storage
 - If-None-Match turns unchanged fetches into 304.
- Propagation bound
 - Worst-case acceptance after revocation = publish lag (5 min) + CDN max-age (5 min) + remaining validity of the artifact the verifier already holds (≤ 24 h CRL, ≤ 8 h OCSP).
 - Decision: validity dominates the bound, not publisher speed
 - Shorten nextUpdate rather than optimize the publisher.
 - Decision: under stapling the OCSP validity window is the attacker's window, because a compromised server can keep stapling its last good response until nextUpdate.

Architecture and flow — IC3 baseline

```

[control plane]                                [data plane / hot path]
Operator / automation
-> Revocation API (compartment authz, Idempotency-Key, 202 + operationId)
-> Ledger DB: issuer_status | revocations | crl_snapshots | outbox | audit
    |
    +-> CRL publisher (per issuer, ordered, debounced)
    |   -> HSM/KMS sign(CRL, crl_key_ref)
    |   -> Object Storage crl/<issuer>/<crl_number>.crl (immutable)
    |   -> verify download -> advance pointer -> alias "latest" -> CDN purge
    |
    |                                     |
    |                                     CDN <- verifiers fetch the CDP URL (CRL)
    |
    +-> OCSP precompute (per issuer, every 1 h)
    |   -> HSM/KMS sign(response, responder_key_ref)
    |   -> Response snapshot (object storage + in-memory on responders)
    |
    |                                     |
    |                                     OCSP responder fleet (stateless, AD-spread)
    |                                     ^               ^
    |                                     TLS servers (fetch + staple)   clients (live OCSP, optional)

Trust: issuer CA cert (KU cRLSign) signs CRLs;
      delegated responder cert (EKU OCSPSigning, <= 30 d) signs OCSP responses.
  
```

- Revocation API owns authorization and the ledger write.

- Mechanism bought: a 202 work request with an operation id, so the caller never waits on signing.
- Cost paid: "revoked" and "published" are two states the API must expose.
- Ledger database owns the ordered truth per issuer.
 - Mechanism bought: a per-issuer generation counter makes "everything through generation g" a well-defined snapshot.
 - Cost paid: one row per issuer serializes revocations
 - Fine at 3 / s, and the reason to reject a global counter.
- CRL publisher owns snapshot building and the pointer.
 - Mechanism bought: immutable objects per CRL number plus a pointer that advances only after verification.
 - Cost paid: an extra download and verify per publish
 - Orphan objects after a crash that a reconciler must tolerate.
- OSCP precompute and responders own per-certificate status.
 - Mechanism bought: responses are signed ahead of time, so the hot path is a memory lookup with no database or HSM dependency.
 - Cost paid: HSM volume scales with certificate count and refresh interval
 - A nonce-requiring client cannot be served from cache.
- Object storage and CDN own distribution.
 - Mechanism bought: static stability
 - A signed CRL is valid without any service being up.
 - Cost paid: max-age and purge latency add to the propagation bound.
- HSM/KMS owns key custody.
 - Mechanism bought: CRL and responder keys never exist in service memory
 - Each signature is audited.
 - Cost paid: throttling or outage stops freshness, not validity
 - The design must alarm on remaining validity.
- Relying parties own the fail policy.
 - Mechanism bought: a published SLO makes hard-fail viable
 - We cannot impose it.
 - Cost paid: uncontrolled clients keep soft-fail, so their bound is the certificate TTL.

→ Operator calls POST /v1/issuers/{issuer}/revocations with an Idempotency-Key.

→ API resolves the tenant from auth context, authorizes the issuer in its compartment, and validates the serial encoding.

→ Transaction: claim the intent, allocate generation = next_generation, insert the revocation row, store the ACCEPTED operation, write outbox and audit rows.

→ Return 202 {revocationId, operationId} with Location: /v1/operations/{operationId}.

→ Publisher consumes the event, debounces 60 s per issuer, and builds the CRL through generation g.

→ HSM signs the DER CRL with the issuer's CRL key; the publisher records the SHA-256.

→ Object is written to its immutable key, re-downloaded, and signature-verified against the issuer certificate.

→ Transaction: insert crl_snapshots(crl_number, through_generation), set published_generation = g, advance the pointer.

→ Alias latest is rewritten and the CDN path purged; OSCP precompute re-signs the affected CertID as revoked first.

→ Operation moves to SUCCEEDED with publicationGeneration = g; verifiers observe revoked on their next fetch.

API and data model — IC3 baseline

Surface	Contract	Conflict/error behavior
POST /v1/issuers/{issuer}/revocations	{serial, reason, invalidityDate?}; I; 202 {revocationId, operationId}	Same serial replays the original 202; different reason on a revoked serial returns 409; unknown issuer 404
GET /v1/issuers/{issuer}/revocations/{serial}	{revokedAt, reason, generation, publicationGeneration, crlNumber?}	Administrative view, not a status protocol; 404 follows tenant visibility
GET /v1/issuers/{issuer}/crl	If-None-Match; application/pkix-crl, ETag, Cache-Control: max-age=300	304 when unchanged; never serves a snapshot the pointer has not reached
POST /v1/issuers/{issuer}/crl-publications	{reason}; I; forces a publish through the current generation; 202 operation	409 if the issuer is disabled; coalesces with an in-flight publish of the same generation
POST /v1/ocsp	DER application/ocsp-request in, application/ocsp-response out	Status lives inside the signed response; malformedRequest / unauthorized are OCSP statuses, not JSON
GET /v1/ocsp/{base64url(request)}	RFC 5019-style cacheable GET; same response as POST	Requests over 1 KB rejected; unknown issuer hash returns unauthorized
GET /v1/operations/{id}	{state, publicationGeneration?}	Shared work-request contract ACCEPTED → RUNNING → SUCCEEDED / FAILED

State	Key	Fields/invariant
Issuer status	(tenant_id, issuer_id)	next_generation, published_generation, crl_key_ref, responder_key_ref, publish interval, validity
Revocation	(tenant_id, issuer_id, serial); UNIQUE (tenant_id, issuer_id, generation)	generation, revoked_at, reason, invalidity_date, not_after copied at revoke time for retention
CRL snapshot	(tenant_id, issuer_id, crl_number)	through_generation, this_update, next_update, object_ref, sha256; CHECK (next_update > this_update)
OCSP response	(issuer_id, serial) resolved from CertID	status, through_generation, this_update, next_update, DER reference
Responder key	(tenant_id, issuer_id, responder_version)	HSM key_ref, responder certificate reference, not_after; two versions overlap during rotation
Issuer hash map	(hash_alg, issuer_name_hash, issuer_key_hash)	Resolves an OCSP CertID to (tenant_id, issuer_id) without any caller-supplied tenant

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - OCSP responses are deliberately not relational
 - Responders serve a precomputed in-memory snapshot keyed by (issuer_id, serial).
 - DER bytes sit on object storage and the CDN.

```

CREATE TABLE issuer_status (
  tenant_id text NOT NULL, issuer_id text NOT NULL,
  next_generation bigint NOT NULL DEFAULT 1,      -- next revocation generation to allocate
  published_generation bigint NOT NULL DEFAULT 0,  -- public pointer; advanced only after a signed snapshot
  exists
  crl_key_ref text NOT NULL,                      -- HSM handle for CRL signing, never key material
  responder_key_ref text NOT NULL,                -- current responder key handle
  publish_interval_s integer NOT NULL DEFAULT 3600,
  crl_validity_s integer NOT NULL DEFAULT 86400,
  PRIMARY KEY (tenant_id, issuer_id),
  CHECK (published_generation < next_generation),
  CHECK (publish_interval_s > 0 AND crl_validity_s > publish_interval_s)
);
CREATE TABLE revocations (
  tenant_id text NOT NULL, issuer_id text NOT NULL, serial numeric(49,0) NOT NULL,
  generation bigint NOT NULL, revoked_at timestampz NOT NULL,
  reason text NOT NULL, invalidity_date timestampz,
  not_after timestampz NOT NULL,                  -- copied at revoke time; retention needs no join into
  issuance
  PRIMARY KEY (tenant_id, issuer_id, serial),
  UNIQUE (tenant_id, issuer_id, generation),
  FOREIGN KEY (tenant_id, issuer_id) REFERENCES issuer_status,
  CHECK (serial > 0),
  CHECK (invalidity_date IS NULL OR invalidity_date <= revoked_at)
);
CREATE INDEX revocations_retention ON revocations (not_after);
CREATE TABLE crl_snapshots (
  tenant_id text NOT NULL, issuer_id text NOT NULL, crl_number bigint NOT NULL,
  through_generation bigint NOT NULL, this_update timestampz NOT NULL,
  next_update timestampz NOT NULL, object_ref text NOT NULL, sha256 text NOT NULL,
  PRIMARY KEY (tenant_id, issuer_id, crl_number),
  UNIQUE (tenant_id, issuer_id, through_generation),
  FOREIGN KEY (tenant_id, issuer_id) REFERENCES issuer_status,
  CHECK (next_update > this_update)
);
CREATE TABLE responder_keys (
  tenant_id text NOT NULL, issuer_id text NOT NULL, responder_version bigint NOT NULL,
  key_ref text NOT NULL,                          -- HSM handle, never key material
  responder_cert_ref text NOT NULL,                -- delegated certificate with id-kp-OCSPSigning
  not_before timestampz NOT NULL, not_after timestampz NOT NULL,
  PRIMARY KEY (tenant_id, issuer_id, responder_version),
  FOREIGN KEY (tenant_id, issuer_id) REFERENCES issuer_status,
  CHECK (not_after > not_before)
);
CREATE INDEX responder_keys_expiry ON responder_keys (not_after);
CREATE TABLE issuer_hash_map (
  hash_alg text NOT NULL, issuer_name_hash bytea NOT NULL, issuer_key_hash bytea NOT NULL,
  tenant_id text NOT NULL, issuer_id text NOT NULL,
  PRIMARY KEY (hash_alg, issuer_name_hash, issuer_key_hash), -- no tenant prefix: resolved from CertID
  alone
  FOREIGN KEY (tenant_id, issuer_id) REFERENCES issuer_status,
  CHECK (hash_alg IN ('SHA1', 'SHA256'))
);

```

- What the DDL does not capture
 - The OCSP snapshot and the issued-serial set replicated from the issuance ledger live in responder memory.
 - CRL DER bytes live on object storage; `object_ref` and `sha256` only point at them.
 - `crl_key_ref` and `key_ref` are HSM handles; no key material is stored.
 - Deviation from the companion
 - `revocations.not_after`, the publish settings on `issuer_status`, `responder_keys`, and `issuer_hash_map` are additions.
 - They let retention sweeps, rotation overlap, and `CertID` routing run without a join into the issuance service.
- Proposed service conventions

- Intent rows use the shared `api_intents` table from the API conventions, retained past the retry horizon.
- `serial` is `numeric(49,0)`, positive, and at most 20 octets DER-encoded
- The SQL type alone does not enforce the X.509 rule.
- Revocation takes no foreign key into the issuance service
- An orphan certificate that was signed but never recorded must still be revocable by serial.
- CDP and AIA URLs are embedded at issuance in [s2.01](#), so they are a frozen contract for every issued certificate.
- Reason codes follow RFC 5280 `CRLReason`
- `keyCompromise` with an `invalidityDate` earlier than `revokedAt` is the reason that field exists.

Revocation ledger and CRL publication — IC3 baseline

- Ordered ledger
 - Allocate an issuer generation and insert the revocation in one transaction.
 - A committed generation identifies a consistent prefix, unlike a wall-clock cutoff.
 - A CRL intent reserves its number and exact ledger prefix before signing.
- Snapshot contents
 - Persist issuer version, CRL number, covered generation, `thisUpdate`, `nextUpdate`, and canonical TBS bytes.
 - Keep unexpired revoked serials.
 - Keep an expired serial until it has appeared on at least one successfully published scheduled CRL after expiry.
 - A fixed one-interval grace cannot prove that condition through a publisher outage.
- Signing
 - Baseline uses the issuing CA key authorized for `cRLSign`.
 - Select the exact issuer key version and verify the completed DER artifact.
 - A separate CRL signer needs the indirect-CRL profile and compatible verifier behavior.

```
PUBLISH(issuer):
  transaction:
    claim current publisher fence
    reserve or resume unique CRL intent(number, covered_generation,
                                         thisUpdate, nextUpdate, exact_TBS, body_hash)
  sign exact_TBS using pinned issuer CRL key
  assemble and verify CRL DER
  put immutable object keyed by issuer, intent_id, and artifact_hash
  fetch stored object; verify actual bytes, issuer, number, and ledger coverage
  transaction:
    require current publisher fence and live lease
    require intent.number > published_head.number
    require intent.covered_generation >= published_head.covered_generation
    verify removal rules against previous published snapshot
    commit snapshot, published_head, audit, and notification outbox
```

- Stable CDP
 - Serve the immutable object selected by the authoritative published head.
 - Bound endpoint and CDN cache age by the declared status-validity policy.
 - Do not let workers unconditionally overwrite `latest.crl` after committing a conditional database pointer.
 - That separate overwrite would let a delayed worker regress the public artifact.
- Crash recovery
 - Before head commit, an uploaded artifact is unreferenced and may be retried or garbage-collected.
 - After head commit, an outbox relay repairs missed notifications.
 - Retrying randomized signing may yield different signature bytes for the same reserved TBS.
 - Verify and publish the exact immutable artifact selected by the winning head.
- Freshness
 - Proposed regular publication is every 1 h with a 60 s event debounce.

- Incident publication bypasses debounce but uses the same fenced path.
- Previously cached signed evidence remains acceptable until its verifier-enforced freshness deadline.
- CDN purge accelerates refresh but is not the correctness proof.

OCSP responder, stapling, and relying-party policy — IC3 baseline

- Request routing
 - `CertID = {hashAlgorithm, issuerNameHash, issuerKeyHash, serialNumber}` (RFC 6960).
 - The issuer hash map resolves the pair to `(tenant_id, issuer_id)`
 - A request never names a tenant.
 - SHA-1 and SHA-256 hashes are both precomputed per issuer because clients differ.
- Status semantics
 - Proposed stronger policy: good requires membership in the issued set and absence from the covered revocation set.
 - RFC OCSP good status alone does not generally prove issuance or path validity.
 - `revoked: revocationTime` and `reason` from the ledger.
 - `unknown`: the issuer is known but the serial is not in the issued set.
 - Never answer `good` for a serial the issuer never recorded
 - The responder holds an issued-serial set replicated from the issuance ledger.
 - Store or snapshot unavailable: answer `tryLater`, never `good`.
 - `good` does not assert validity dates, path validity, or hostname
 - That stays with the verifier.
- Precomputed responses
 - A batch job signs one response per unexpired certificate per issuer every 1 h with `nextUpdate = thisUpdate + 8 h`.
 - Responders load the snapshot into memory keyed by `(issuer_id, serial)` and serve DER bytes without signing.
 - The `GET` form follows RFC 5019 so a CDN can cache responses with `max-age` bounded by `nextUpdate`.
 - Revoked serials are re-signed first after each publish so the negative propagates before the routine refresh.
- Nonce policy
 - Baseline ignores request nonces and returns the precomputed response
 - This is the lightweight-profile behavior and the reason responses are cacheable.
 - Nonce-required clients need a separately supported live-signing profile or an explicitly negotiated alternative.
 - Do not silently redirect them to a non-nonce protocol and call it equivalent.
 - A live-signed nonce path exists only as a rate-limited stretch item.
- Responder key custody
 - A delegated responder certificate carries EKU `id-kp-OCSPSigning`, is signed by the issuing CA, and is valid ≤ 30 d.
 - The private key lives in the HSM under `responder_key_ref`
 - The batch job calls `sign` and never reads the key.
 - The `nocheck` extension asks clients to omit responder-certificate revocation checks.
 - Bound that exposure with short responder validity and controlled rollover.
 - Rotation keeps two responder versions overlapping so a snapshot signed by the previous key stays verifiable until its `nextUpdate`.
- Stapling
 - Servers we control fetch their own response every 1 h and present it in the TLS `status_request` extension.
 - The server keeps the last good response on disk, so a control-plane or responder outage does not break handshakes until that response reaches `nextUpdate`.
 - Servers alarm when remaining staple validity drops under 2 h.

- Must-staple (RFC 7633) turns a missing staple into a hard failure and is enabled per certificate profile, never globally.
- Soft-fail versus hard-fail
 - Soft-fail: the verifier accepts when status is unreachable
 - Revocation becomes advisory and the bound is the certificate TTL.
 - Hard-fail rejects when no fresh acceptable status evidence exists.
 - A cached fresh CRL or staple can avoid a live responder dependency until its deadline.
 - Choose fail policy per relying-party profile.
 - Strict internal clients require fresh status evidence and fail closed when none remains.
 - Explicit soft-fail profiles accept a longer exposure window for unavailable status.
- Propagation bound stated aloud
 - Stapled server, uncompromised: $\leq 1\text{ h}$ refresh plus 5 min publish lag.
 - Stapled server whose key is stolen: $\leq 8\text{ h}$, the validity of the last `good` response the attacker holds.
 - A strict CRL verifier can accept previously fresh evidence only through `nextUpdate` plus its declared clock allowance.
 - A fetch interval cannot extend `nextUpdate` unless the profile explicitly allows stale evidence.
 - A live-OCSP client may retain a previously good response through that response's accepted validity window.
 - A five-minute publisher or CDN refresh does not erase an attacker's still-valid signed response.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Ledger database unavailable (control-plane outage)	New revocations and publications	Health checks, 503 rate	CRLs and precomputed responses stay valid to <code>nextUpdate</code> ; API returns 503 with <code>Retry-After</code> ; responders never query the ledger	Fail over the replica; publisher resumes from <code>published_generation</code>
Publisher crashes mid-publish	One issuer's freshness	Pointer lag; <code>nextUpdate - now</code> alarm	Previous CRL keeps serving	Rerun <code>PUBLISH</code> at the same <code>cr1_number</code> ; <code>put_if_absent</code> makes it idempotent
HSM or KMS throttled or down	Every fresh signature	Signing error rate, queue age	Validity countdown; nothing becomes invalid early	Prioritize CRLs over OCSP batch; page at 50% validity consumed
CDN serves stale CRL after a failed purge	Verifiers behind that edge	Canary fetch compares <code>ETag</code> with the pointer	<code>max-age=300</code> caps staleness at 5 min	Re-purge; verifiers can use the numbered object URL
OCSP responder fleet overloaded	Live-OCSP clients only	p99 latency, 5xx	Stapled servers unaffected; shed unknown-issuer requests first	Scale stateless responders; CDN caches <code>GET</code> responses
Responder certificate expires	Every OCSP response from that issuer	Remaining-validity gauge	Verifiers reject responses; hard-fail servers stop handshakes	Auto-rotate at 50% life with two overlapping responder versions
Mass revocation spike ($10\text{k} / \text{h}$)	One issuer's CRL size and precompute backlog	Generation rate, CRL bytes	Debounce coalesces publishes; revoked serials re-signed before routine refresh	Per-tenant revocation rate quota; large-CRL guard
Bad object or wrong signing key	One issuer's verifiers	Post-write download and verify fails	Pointer never advances; old CRL keeps serving	Rebuild with the correct <code>cr1_key_ref</code> ; audit the key reference change
Revocation row dropped by mistake	Would turn revoked into good	Additive-diff guard and audit comparison	Publish blocked for that issuer	Restore the row from the audit log; republish

- Observe
 - Minimum `nextUpdate - now` per issuer for CRL and OCSP snapshots.
 - Publication lag from revocation generation to `published_generation`.
 - Oldest precomputed response age and responder certificate remaining validity.
 - CRL bytes per issuer and additive-diff guard trips.
 - OCSP status mix (`good`, `revoked`, `unknown`, `tryLater`), CDN hit ratio, HSM signature failures.
- Retry discipline
 - Revocation clients retry the same `Idempotency-Key` with exponential backoff and jitter.
 - The publisher retries the same `crl_number`
 - A new number is allocated only after a verified failure of the old object.
 - HSM throttling is retried with jitter and a bounded elapsed deadline
 - Freshness alarms cover the remainder.
 - OCSP clients honor `Retry-After` on `tryLater`
 - Nothing in the system retries its way to `good`.
- Tenant isolation
 - An issuer belongs to a compartment
 - A policy of the documented OCI shape `allow group X to manage certificate-authority-family in compartment Y` gates revocation and forced publish.
 - CRL objects and OCSP snapshots are per issuer
 - Nothing aggregates across tenants.
 - OCSP routing uses issuer hashes, so a request cannot select another tenant's issuer by name.
 - A tenant's token receives `404` on another tenant's issuer, not `403`, to avoid existence leaks.
 - Revocation and forced-publish rates are quota-limited per tenant so one incident cannot starve HSM capacity for others.
- Rollout and rollback safety
 - A DER-builder change is shadow-built and verified with an independent parser before the pointer moves.
 - Responders must load the previous and the new snapshot format during a deploy.
 - CDP and AIA URLs never change
 - A URL change would orphan every issued certificate.
 - Canary responders behind the load balancer
 - Rollback is the pointer staying at the last verified snapshot.
 - Schema changes follow expand, migrate, contract
 - `revocations` never gains a `DELETE` path.
- Documented OCI behavior versus proposed design
 - Documented: OCI Certificates lets a CA publish its CRL to an Object Storage bucket configured on the CA, and revocation is a CA-level operation (OCI Certificates overview below).
 - Proposed: the generation ledger, publish-then-advance pointer, additive-diff guard, and the OCSP responder are this answer's design.
 - The OCSP responder here is a proposed component.
 - This answer does not claim OCI Certificates provides that managed endpoint.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Full CRL	Signed list of unexpired revoked serials at a CDP URL	Offline-verifiable, CDN-cacheable, no handshake dependency	Staleness equals validity; bytes grow with revocations	Baseline for every issuer; mandatory for appliances
Delta CRL	Signed list of changes since a base CRL number	Small frequent updates	Two artifacts to keep consistent; patchy verifier support	Base CRL past ≈ 10 MB and verifiers that support it

Option	What it is	Strength	Weakness	Choose when
Live-signed OCSP	Responder signs each answer on request	Fresh; nonce support	HSM on the hot path; responder is a tier-0 dependency	Few controlled clients that require nonces
Precomputed OCSP plus stapling	Batch-signed responses served from memory and stapled by servers	No client-to-CA call; hard-fail is viable	Validity window is the compromise window; signing scales with cert count	Servers we control; baseline OCSP shape
Short-lived certificates	TTL of hours to days; expiry is the revocation	No status infrastructure on the verifier path	Issuance becomes a hot path; clock skew matters; every consumer must auto-renew	Automated internal mTLS; the honest control for soft-fail clients
Push filters (CRLite-style)	Vendor pushes a compact set of all revocations	Fresh with no per-site fetch	Only one party controlling the verifier fleet can do it	Mention only; not applicable to customer verifiers

- IC4 stretch — stronger propagation bound
 - Requirement: $\leq 5 \text{ min}$ for the controlled fleet after a compromise.
 - A five-minute compromise target requires every accepted good-status lease to expire within that budget, including skew and processing delay.
 - A 15-minute OCSP response cannot by itself meet a five-minute bound.
 - Add a signed revocation delta feed on the configuration channel of [s3.03](#) so sidecars learn of a revocation without polling.
 - State the limit: uncontrolled clients stay bounded by the certificate TTL, so the profile also moves to short-lived leaves.
- IC4 stretch — 10× load
 - 10M certificates at a 1 h refresh is $\approx 2,800$ signatures per second
 - Shard precompute per issuer across HSM partitions.
 - Sign lazily for certificates with no observed consumer in 24 h
 - Precompute only the active set.
 - Move issuers past 10 MB to delta CRLs or partitioned CRLs scoped by the Issuing Distribution Point extension.
 - Responders stay stateless
 - The snapshot grows, the read path does not change.
- IC4 stretch — cost and blast radius
 - HSM signature volume is the dominant cost
 - The OCSP refresh interval and the active-set filter are the cost knobs.
 - Per-issuer objects and snapshots make a bad publish affect one issuer
 - The additive-diff guard catches a fleet-level bug per issuer.
 - Responders are cells per availability domain
 - A cell outage degrades live OCSP in one domain and never touches stapled servers.
- IC4 stretch — migration to must-staple
 - Inventory verifiers and their fetch telemetry before enabling any hard-fail.
 - Enable must-staple per certificate profile and cohort, with a rollback path that reissues without the extension.
 - Treat the responder SLO as a published contract from that point.
- Weakest assumption
 - Verifiers refresh before `nextUpdate` and appliances fetch the CRL at all
 - A wallet-based client that never fetches has the certificate TTL as its bound, and revocation is theater for it.
 - Mitigation: per-consumer fetch telemetry from CDN logs, a longer-TTL legacy profile that carries a CRL it actually fetches, and short-lived certificates for everything automated.
- Over-engineering warning
 - Do not build a live-signed nonce path for internal mTLS

- Precomputed responses plus stapling cover the controlled fleet.
- Do not build delta CRLs for a 0.5 MB list.
- Do not build a global CRL
- It couples every tenant's freshness and blast radius.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Relying-party cohorts, propagation bound, soft-fail versus hard-fail named	Splits the bound by cohort and states which cohort cannot be bounded	Treats the fail policy as a requirement, not an implementation detail
Estimates	Revocation rate, CRL bytes, OCSP signing load	Shows that validity, not publisher speed, dominates the bound	A number changes nextUpdate, not just fleet size
Architecture	Control-plane ledger, publisher, object storage plus CDN, responders off the database	States the cost of each split and where static stability comes from	Every component has a mechanism and a price
Deep dive	Generation ledger, publish-then-advance, good never from an outage	Additive-diff guard, responder key custody, the attacker's window under stapling	Reasons about what a cached artifact still says after the truth changed
Operations	Failure table, freshness alarms, HSM custody, tenant quotas	Rollout of must-staple, per-issuer blast radius, cost knobs	Volunteers degradation and rollback before being asked
Wrap	Recap and the CRL-versus-OCSP tradeoff	Names short-lived certificates as the honest control and the weakest assumption first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Revocation is urgent
 - Why return 202 instead of publishing synchronously?
 - **IC3 baseline**
 - The durable ledger write is synchronous and ordered
 - Signing and distribution are asynchronous and idempotent, and the operation exposes the publication generation.
 - **IC4 stretch**
 - Forced publish bypasses the debounce, but the bound is still the validity of artifacts already cached, so a synchronous publish would not shorten it.
- Q2: The database is down and the responder cannot find the serial.
 - Return good?
 - **IC3 baseline**
 - Never
 - good comes only from a signed snapshot with a known generation, and an outage answers tryLater.
 - **IC4 stretch**
 - Responders serve from memory and never touch the ledger, so an outage changes freshness, not answers.
- Q3: A customer deletes the certificate resource.
 - Is it revoked?
 - **IC3 baseline**
 - No
 - The public certificate stays valid until notAfter unless a revocation is on a signed CRL or OCSP response.
 - **IC4 stretch**

- Deletion can trigger a revocation with reason `cessationOfOperation`, and the revocation row outlives the deleted resource.
- Q4: Why not set CRL `nextUpdate` to five minutes and be done?
 - **IC3 baseline**
 - A five-minute validity makes every publisher or CDN hiccup a fleet outage for hard-fail verifiers.
 - **IC4 stretch**
 - Shorten validity only where the staple already holds a local copy
 - Keep 24 h for appliances that fetch hourly at best.
- Q5 (premise reversal): Why have OCSP at all?
 - Short-lived certificates make revocation unnecessary.
 - **IC3 baseline**
 - For automated consumers that is the honest answer, and short TTL is the primary control for them.
 - **IC4 stretch**
 - CRL stays for appliances that cannot renew hourly and stapling stays for public endpoints
 - The deciding variable is whether every consumer can renew automatically.
- Q6: The attacker holds the stolen key and keeps stapling the last `good` response.
 - **IC3 baseline**
 - Correct, and that is why OCSP validity is the compromise window
 - 8 h is the stated bound.
 - **IC4 stretch**
 - Shorten the window for high-value profiles and pair it with short-lived leaves so the certificate itself expires inside the window.
- Q7: Can any key in the HSM sign the CRL?
 - **IC3 baseline**
 - Use the issuer's `cRLSign`-authorized key in the baseline.
 - An alternative signer needs correctly encoded indirect-CRL scope and verifier support, not merely a certificate with `cRLSign`.
 - **IC4 stretch**
 - A dedicated CRL-signing key limits use of the issuing key and rotates with overlap like the responder key.
- Q8: An intermediate is compromised and ten thousand leaves must go.
 - Where is the intermediate revoked?
 - **IC3 baseline**
 - On its parent's CRL, so the root's CRL path must exist and be fetchable
 - Leaves go on the intermediate's own CRL in a few coalesced publishes.
 - **IC4 stretch**
 - Replace the compromised issuer through an uncompromised trust path and verify adoption.
 - Cross-signing by the compromised authority does not restore trust.
- Validation scenarios
 - Same revocation submitted twice with the same key.
 - Two revocations for one issuer committed out of timestamp order.
 - Publisher crash after object write and before the snapshot row.
 - Two publishers racing for the same issuer.
 - A new CRL that would drop an unexpired serial.
 - OCSP request for a serial the issuer never issued.
 - Responder snapshot unavailable during a store outage.
 - Responder certificate within 24 h of expiry.
 - CDN purge failure with a 5 min `max-age`.

- Control-plane outage beyond the 8 h staple lifetime: strict clients reject unless another still-valid acceptable status source exists.

Recall card

- "Revocation is an ordered generation per issuer; a CRL is a signed snapshot through one generation."
- "Publish, verify, then advance the pointer; never the other way round."
- "A store outage answers `tryLater`; `good` only ever comes from a signed snapshot."
- "The propagation bound is the validity of what verifiers already hold, not how fast we publish."
- "Short-lived certificates are the honest control for soft-fail clients; CRL and stapled OCSP cover the rest."
- Wrap-up script
 - "The design is an append-only revocation ledger with a per-issuer generation, feeding signed CRL snapshots on object storage plus CDN and precomputed OCSP responses served from memory, with every key in the HSM."
 - "The key invariant is publish-then-advance: a snapshot is written and verified before the pointer moves, and a new CRL may only drop expired serials."
 - "The main tradeoff is validity versus availability: a shorter `nextUpdate` bounds a compromise faster but turns every freshness failure into a hard-fail outage."
 - "My weakest assumption is that verifiers actually fetch before `nextUpdate`; I would validate fetch telemetry per consumer cohort before promising any propagation bound."

References

- [Ranked question and report provenance](#).
- [OCI role framing](#).
- [Detailed API and schema companion](#).
- [Shared API conventions](#).
- Reference check: 2026-09-09.
 - Official documentation and RFCs support the mechanisms cited next to each link.
 - Capacity values, validity windows, and architecture choices remain proposed interview assumptions.
- [RFC 5280](#): CRL profile, `thisUpdate` / `nextUpdate`, CRL Number, `cRLSign`, reason codes, and the rule that an entry stays until one scheduled CRL after expiry (§3.3, §5).
- [RFC 6960](#): OCSP `CertID`, `good` / `revoked` / `unknown`, `tryLater`, delegated responder with `id-kp-OCSPSigning`, `id-pkix-ocsp-nocheck`, nonce extension.
- [RFC 5019](#): lightweight OCSP profile
 - `GET` requests, precomputed responses, HTTP caching bounded by `nextUpdate`.
- [RFC 8954](#): OCSP nonce update
 - Why nonce-requiring clients cannot use cached responses.
- [RFC 6066](#) and [RFC 7633](#): TLS `status_request` (stapling) and the TLS Feature extension (must-staple).
- [OCI Certificates overview](#): documented CA hierarchy, revocation, and CRL publication to Object Storage.
- [OCI Vault key overview](#): HSM-backed keys and the sign operation that keeps CRL and responder keys in custody.
- [OCI IAM policies](#): compartment-scoped policy shape used for tenant isolation.
- [PKI and KMS mechanism reference](#): revocation options compared (A.6), soft-fail dilemma, CRL publisher failure modes (C.6).
- [Multi-tenant certificate authority](#): issuer hierarchy and serial namespace this design revokes against.
- [Renewal and CA rollover](#): old-key CRL signing after rotation
 - Intermediate compromise remediation.
- [Async control-plane resource API](#): the 202 work-request contract reused by revocation and forced publish.